

第6章 图算法

6.1 基本概念（前置知识）

- 机内表示：邻接表
- 默认区分有向图和无向图，不考虑自环和重边
- 无向图只有度的概念，有向图区分入度和出度的概念
- 道路与回路，简单道路与简单回路
- 连通图，连通分支
- 树，森林
- 子图，生成树
- 偶图（二分图）
- 加权图
- 欧拉图与欧拉半图，欧拉回路与欧拉道路
- 哈密顿图与哈密顿半图，哈密顿回路与哈密顿道路

6.2 图的遍历

6.2.1 DFS(深度优先搜索)

- **思想**：从任意结点开始，标记其访问过，然后任选与之相邻且未被访问的顶点，递归调用；若找不到与当前结点相邻且未被标记的结点则递归终止，时间复杂度 $\Theta(|V| + |E|)$
- **用途**：求连通子图个数；给图的顶点编号；建立DFS生成树
- **无向图的DFS树**：非树边都是跨代边（在DFS树中连接具有祖孙关系的结点）
- **有向图的DFS树**：非树边包括
 - 后退边：由后代结点指向祖先结点
 - 前向边：由祖先结点指向后代结点
 - 交叉边：关联的两个结点无祖孙关系，但在树的生成过程中，由后访问的结点指向先访问的结点
- **闭链**：除回路的首尾结点重合外，不含重复结点和重复边的回路
在有向图中，有向闭链还要求且边的方向与有向图的方向一致
有向图DFS后有后退边 $\Leftrightarrow$ 有向图含有有向闭链

6.2.2 BFS(广度优先搜索)

- **思想**：从任意结点开始，标记其访问过，然后访问它的所有儿子结点，访问所有儿子结点后，选择任意访问过的儿子结点递归调用；直到遍历全图，时间复杂度 $\Theta(|V| + |E|)$
- **BFS树的性质**

- 树中的父结点一定比所有儿子结点更早访问
- 无权图中，根结点到所有结点的最短路径为BFS树中结点的层次
- 非树边连接的结点，层次数至多相差 1

6.3 拓扑排序

- **问题：**称不含有向闭链的有向图为有向非循环图，为有向非循环图中的结点按 $1..|V|$ 编号，要求若存在边 $\langle v, w \rangle$ 则结点 v 的标记小于结点 w
- **BFS：**必然能找到入度为 0 的结点 r 并标记为 1，之后删除该结点及相关联的边，并对新图递归调用，时间复杂度 $\Theta(|V| + |E|)$

6.4 最短路径问题

6.4.1 无权有向图的单源最短路径

- **问题：**给定无权有向图 $G = (V, E)$ ，满足从 $v \in V$ 出发能到达所有结点，求从 V 出发到达所有结点的最短路径
- **归纳：**可对该图进行拓扑排序，设已经知道 $1..n - 1$ 的最短路径长度 $l[1..n - 1]$ ，则对 n 只要找到所有存在边 $\langle k, n \rangle$ 的结点 k ，并取 $\min_k l[k] + 1$ 即可
- **实现：**拓扑排序中删除结点 v 时，设删除 $\langle v, w \rangle$ ，设从源头到 v 的路径长为 l ，则可将暂存的源头到结点 w 的路径长最小值与 $l + 1$ 比较，并保留更小的值，可在与拓扑排序相同阶数的时间内，求解无权有向图的单源最短路径问题

6.4.2 正权有向图的单源最短路径

- **问题：**给定加权有向图 $G = (V, E, W)$ ，满足从 $v \in V$ 出发能到达所有结点，且所有边的权值均为正值，求从 V 出发到达所有结点的最短路径
- **Dijkstra算法：**将所有结点的最小距离初始化为 ∞
更新从 v 出发经过 1 条边可达的结点的最小距离，并最终确定上述更新的最小距离中，距离的最小值 $l[x]$
再更新从 x 出发经过 1 条边可达的结点的最小距离，并最终确定上述全部更新的最小距离中，距离的最小值 $l[y]$
直到最终确定全部结点的最短路径
时间复杂度 $O((|V| + |E|) \log |V|)$

6.4.3 任意权有向图的单源最短路径

- **问题：**给定加权有向图 $G = (V, E, W)$ ，满足从 $v \in V$ 出发能到达所有结点，允许边的权值为正值或负值或 0，求从 V 出发到达所有结点的最短路径
- **Bellman-Ford算法：**将所有结点的最小距离初始化为 ∞
对所有边 $\langle u, w \rangle$ 进行松弛操作：若 $dis[u] + W[\langle u, w \rangle] < dis[w]$ ，则更新 $dis[w]$ 为

$$dis[u] + W[\langle u, v \rangle]$$

更新 $|V| - 1$ 次即可得到任意权有向图的单源最短路径

时间复杂度 $O(|V||E|)$

6.4.4 任意图的全部最短路径

- **问题**：给定加权任意图 $G = (V, E, W)$ ，求任意结点对之间的最短路径
- **对边归纳**：归纳过程中，每次加入新的边时，检查每对结点之间是否有包含新加入边的最短路径，时间复杂度 $O(|E||V|^2)$
- **对顶点归纳**：归纳过程中，每次加入新的结点时，求出新结点到其他顶点的最短路径，还要检查其他结点对之间是否存在经过新结点的更短路径，时间复杂度 $O(|V|^3)$
- **Floyd算法**：按路径类型归纳，从 1 到 n 标记各顶点，设从 u 到 w ，且路径上最大结点恰好为 k 的路径为 $u - w - k - path$ ，若 u 和 w 之间存在边，成为 $u - w - 0 - path$
 归纳过程中，已知 $u - w - k - path$ 的最短长度，求 $u - w - m - path$ 的最短长度 ($m > k$)，则结点 m 必须包含在 $u - w - m - path$ 中，则 $u - v - m - path$ 的最短长度为 $u - m - i - path$ 的最短长度与 $m - w - j - path$ 最短长度之和 ($i < m, j < m$)
 时间复杂度 $O(|V|^3)$ ，但单步操作简单，可实现性强

6.4.5 最短路径算法应用：负环问题

- **定义**：有向图中的边权和为负值的有向回路
- **Bellman-Ford算法**：更新第 n 次时，若还有更新，则有负环
- **Floyd算法**：若求出 $v - v - i - path$ 的最短长度 < 0 ，则有负环

6.5 传递闭包问题

- **问题**：有向图 $G = (V, E)$ ，求有向图 $C = (V, F)$ ，满足：若 G 中存在 $v \rightarrow w$ 的路径，则 $\langle v, w \rangle \in F$
- **归约的应用**：将 G 扩展为 G' ，满足： $V(G) = V(G')$ ； G' 为有向完全图；若 $e \in E(G)$ ，则 G' 中 $w(e') = 0$ ，否则 $w(e') = 1$
 对 G' 求解最短路径问题，若 $v \rightarrow w$ 的最短路径长为 0，则 $\langle v, w \rangle \in F$ ，否则 $\langle v, w \rangle \notin F$
- **归约**：将问题 A 转化为问题 B ，解决问题 B 并通过某种方式变为问题 A 的解，从而求解问题 A
 有关归约的问题将在第9章和第10章进一步讨论

6.6 最小生成树

- **问题**：给定无向加权连通图 $G = (V, E)$ ，求边权值和最小的生成树 T
- **对图边归纳**：原图的边数从 $< m$ 到 m 时问题可能不一致，图也可能变得不连通
- **对树边归纳**：找到了 k 条边的最小生成树 T ，如何扩展到 $k + 1$ 条边的最小生成树 T'
 - **Kruscal算法**：不断加入未遍历到的最短边，若加入边时形成了回路，则去掉该边，否则将该边加入生成树，直到遍历所有边

- **Prim算法**：从任意结点开始，加入与已生成的树相关联的最短边，若形成了回路则去掉该边，直到遍历所有边

6.7 图的分解

6.7.1 双连通分支

- **定义**：无向图中，若边集相关联的结点中，每对结点之间至少存在 k 条不相交（没有重复结点和重复边）的路径，且该边集加入任意边后均不再满足以上条件，则称该**边集**为 k 连通分支
若图仅包含 1 个 k 连通分支，则称该图为 k 连通图
- **Menger定理**： $G = (V, E)$ 无向连通，若通过删除结点使得 u, v 不连通，则删除结点数目至少为 u, v 之间不相交路径的最大数目
- **Whitney定理**：图是 k 连通的 $\Leftrightarrow$ 使图不连通至少要删除 k 个结点
- **关节点**：图不是双连通的 $\Leftrightarrow$ 图中存在结点 v ，删除 v 后图不再连通，称结点 v 为关节点
- **引理**： e, f 属于同一双连通分支 $\Leftrightarrow$ 存在闭链 C 使得 $e, f \in C$
若将连接双连通分支的边也视为双连通分支，则 e 属于仅属于单个双连通分支
- **问题**：将图划分为极大的双连通分支
- **双连通树**：将图的双连通分支抽象为结点，若在图中双连通分支之间有边相连，则在新图中对应的结点之间有边相连，则新图一定是树，称为双连通树（可按照BFS的顺序构造）
根据双连通分支的性质，在对边归纳的过程中，若新加入的边在双连通树中形成了回路，则可将回路中的结点合并为更大的双连通分支
查找回路复杂度 $\Theta(|V|)$ ，边数 $|E|$ ，则时间复杂度 $O(|V||E|)$
- **DFS树与双连通分支**：
 - 叶结点一定不是关节点
 - 根节点在只有 1 棵子树时等价于叶结点，多于 1 棵子树时一定是关节点
 - 对于内部结点，若找不到非树边将结点 v 的子孙连接到 v 的祖先，则 v 为关节点
- **DFS优化双连通分支划分**：
 - **最高祖先**定义为，从 v 出发，只经过后代和非树边，到达的DFS层次数最低的结点 w ，称 w 为 v 的最高祖先（ v 的最高祖先也有可能是 v 本身）
 - v 的最高祖先是 v 后代的最高祖先中DFS层次数最低的结点，或 v 关联的非树边能到达的DFS层次数最低的结点
 - v 是关节点 $\Leftarrow v$ 存在后代 w ， w 的最高祖先不高于 v
- **DFS划分双连通分支算法实现**：
 - DFS到达结点 v 时，对 v 未被遍历的儿子递归调用DFS，找到 $High$ 值表示最高祖先的层次数（根节点为 0），待所有后代结点的DFS调用完成后，计算 $High(v)$ ，同时判断关节点
 - 时间复杂度 $O(|V| + |E|)$

6.7.2 强连通分支

- **定义**：有向图中，若点集满足每对结点之间均有路径，且该点集加入任意结点后均不再满足以上条件，称该点集为强连通分支

- 引理：两个结点属于同一双连通分支 $\Leftrightarrow$ 图中存在包含这两个结点的有向回路
- 推论：将强连通分支压缩为结点后，图为有向无环图
- 引理：每个顶点属于且仅属于 1 个强连通分支
- **DFS划分**：进入结点时，操作数 $+1$ ，并记为该结点的进入值 $pre[v]$ ；离开结点时，操作数 $+1$ ，并记为该结点的离开值 $post[v]$ ，则任意两对结点的(进入值，离开值)区间 $(pre[v], post[v]), (pre[u], post[u])$ 只有嵌套或分离情况
 - 若 $\langle u, v \rangle$ 存在，则根据 $(pre[v], post[v]), (pre[u], post[u])$ 的关系，可判断 $\langle u, v \rangle$ 的类型
 - $(pre[u], post[u]) \supset (pre[v], post[v]) \Leftrightarrow \langle u, v \rangle$ 为前向边
 - $(pre[u], post[u]) \subset (pre[v], post[v]) \Leftrightarrow \langle u, v \rangle$ 为后退边
 - $(pre[u], post[u]) \cap (pre[v], post[v]) = \emptyset \Leftrightarrow \langle u, v \rangle$ 为交叉边
- **凝聚图**：将强连通分支压缩为结点，若原图中强连通分支之间有边相连，则新图中对应结点之间有边相连，称新图为凝聚图，则凝聚图是有向无环图

凝聚图存在入度为 0 的结点，称为源点，对应原图的源点强连通分支；存在出度为 0 的结点，称为汇点，对应原图的汇点双连通分支

若以汇点强连通分支中的任意结点为根调用DFS，则DFS遍历该汇点强连通分支的所有结点后结束，不会涉及其他双连通分支
- **post 的性质**：DFS中 $post$ 值最大的顶点一定位于某个源点强连通分支

强连通分支之间只存在单向路径，否则可将多个结点对应的点集压缩为更大的强连通分支

DFS从强连通分支 A 到达强连通分支 B 后，至少会遍历强连通分支 B ，再回到强连通分支 A ，则 A 中部分结点的 $post$ 值一定大于 B 中所有结点的 $post$ 值，故有如上性质
- **算法总结**：将全图反转，寻找反转图重点源点强连通分支，对应正图中的汇点强连通分支，在正图中遍历该强连通分支后去掉该子图，从而处理更小规模的问题

6.7.3 图分解的应用

- **无向图中偶数长度闭链**：必然包含在双连通分支中

边数多于 1，且含有多于 1 个奇数长度闭链的双连通图，必然存在长度为偶数的闭链
- **有向图中奇数长度闭链**：必然包含在强连通分支中

有向图存在奇数长度闭链 $\Leftrightarrow$ 以任意结点为根进行DFS，交替标记结点 (i.e. $\langle u, v \rangle$ 存在时，若 u 的标记为 1，则 v 标记 0；若 u 的标记为 0，则 v 标记 1)，过程中会发现一条边关联的 2 个结点的标记已经相同

6.8 匹配问题

- **定义**：图边集的子集，满足其中任意边都没有公共顶点
- **完美匹配**：匹配的边关联图中的所有结点
- **无匹配**：结点在匹配中没有关联的边
- **最大匹配**：具有最大边数的匹配
- **极大匹配**：匹配加入任意边后，均不满足任意边没有公共顶点

6.8.1 稠密图匹配

- **问题：**给定无向图 $G = (V, E)$, $|V| = 2n, \forall v \in V, d(v) \geq n$, 证明必然存在完美匹配, 并设计算法求完美匹配
- **证明与算法：**先寻找任意极大匹配, 方法为: 遍历全图, 不断向已有边集加入边, 判断是否有重复顶点, 可找到边集 M , 设 $|M| = m$, 若 $m = n$ 则为完美匹配
若 $m < n$, 则 $\exists v_1, v_2$ 不与 M 中的边相关联且 $\langle v_1, v_2 \rangle \notin E(G)$, 由于 $d(v_1) \geq n, d(v_2) \geq n$, 从 v_1, v_2 至少发出 $2n$ 条边, 且这 $2n$ 条边的另一个结点均与 M 中的边相关联
 $\therefore |M| < 2n \therefore \exists u_1, u_2, \langle u_1, v_1 \rangle, \langle u_1, v_2 \rangle, \langle u_2, v_1 \rangle, \langle u_2, v_2 \rangle$ 至少存在 3 条边
对边集 M 进行操作: 移除 $\langle u_1, u_2 \rangle$, 加入 $\langle u_1, v_1 \rangle, \langle u_2, v_2 \rangle$ 或 $\langle u_1, v_2 \rangle, \langle u_2, v_1 \rangle$, 则 M 仍然满足匹配且 $|M|++$, 直到 $m = n$ 即找到了图的完美匹配

6.8.2 偶图匹配

- **问题：**设 $G = (U, V, E)$, 其中 U, V 且 $U \cup V = \emptyset$, E 中的边满足: $\{u, v\} \in E \Rightarrow u \in U, v \in V$, 求偶图的最大匹配
- **贪心法：**先匹配度数较小的点, 但不能保证最优解
- **优化法：**
 - **交互路径：**设 $v \in V, u \in U, u, v$ 之间有路径, 且 u, v 均未在 M 中被匹配, 称该路径为交互路径
出现交互路径时, 可将交互路径中, 已匹配的边与未匹配的边互换, 实现 $k \rightarrow k + 1$ 条边的优化
 - **交互路径查找：**设已有边集为 M , M 中的边从 U 到 V , $\neg M$ 中的边从 V 到 U , 可将无向图转变为有向图, 从未匹配的结点开始, 通过图搜索算法DFS/BFS, 可找到交互路径
算法的时间复杂度 $O(|V|(|V| + |E|))$
 - **改进：**从未匹配的顶点开始BFS, 找到未匹配的叶结点, 树中从根结点到叶结点的路径即为交互路径, 并删除路径中的结点和边, 直到找不到交互路径, 时间复杂度 $O(\sqrt{|V|}(|V| + |E|))$

6.9 非常稠密图的哈密顿回路

- **问题：**无向图 $G = (V, E)$ 满足: $\{v, w\} \notin E \Rightarrow d(v) + d(w) \geq n$, 求 G 中的哈密顿回路
- **反向归纳法：**设 $G + \{u, v\} = G'$, 且 G' 中找到了哈密顿回路
 - 若 $\{u, v\}$ 不在上述哈密顿回路中, 则不影响解的情况
 - 若 $\{u, v\}$ 在上述哈密顿回路中, 根据 $d(u) + d(v) \geq n$, 从 u, v 至少发出 n 条边, 而 G 除去 u, v 外只有 $n - 2$ 个结点, 所以存在哈密顿回路中的边 (x_i, x_{i+1}) 满足:
 $(x_i, v), (x_{i+1}, u) \in E(G)$, 可将原回路修改为 $(u, \dots, x_i, v, \dots, x_{i+1}, u)$, 依然为哈密顿回路
时间复杂度 $O(n^2)$